

HelixConstitution

HelixConstitution

Tagline: The universal engineering constitution every project inherits — anti-bluff law, enforced mechanically, shared as one Git submodule.

Summary: HelixConstitution is the single, project-agnostic rulebook — added as a Git submodule by every Helix/vasic-digital project — that encodes non-negotiable engineering discipline (anti-bluff, evidence-only validation, data/host safety, documentation and test coverage) and propagates it to a fleet of 140+ repositories. It is the governance backbone that makes the whole family coherent.

Short description: A universal, inheritable Constitution shipped as a Git submodule. It defines mandatory, non-negotiable rules — anti-bluff evidence gates, false-positive immunity, data and host safety, coverage and documentation discipline — that every consuming project inherits automatically and may extend but never weaken.

Long description: HelixConstitution is the canonical source of truth for the engineering practices shared across every project that opts in by adding it as a Git submodule. Its centrepiece — `Constitution.md` — is an ~1 MB, continuously-versioned document of numbered clauses (the §11.4.x covenant family, currently through §11.4.170) plus per-agent operating manuals (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md`) that import it by reference. Inheritance is three-layer: the universal base (this submodule), the project layer (a project's own `Constitution/CLAUDE/AGENTS` that extend it), and an optional per-subdirectory layer; project rules may tighten but never weaken the base. The document is deliberately domain-agnostic: anything mentioning a specific vendor, hardware SKU, port, or library version must move to the consuming project's own Constitution, and universality has to be *earned* against a four-part test. Its philosophical spine is anti-bluff (§1.1 false-positive immunity, §11.4 end-user quality covenant, §11.4.6 no-guessing, §11.4.69 positive-evidence taxonomy): the bar for shipping is never “tests pass,” but “users can use the feature,” and every green result must cite captured physical

evidence. A companion submodules-catalogue.md (142 repos) enforces catalogue-first, extend-don't-reimplement discovery. Helper scripts locate the submodule from any depth and fan every commit out to four Git providers.

Why we built it: Multiple large product apps and dozens of decoupled reusable submodules, authored by the same owner, kept re-deriving the same hard-won rules — and kept hitting the same failure class: tests and status reports that claim success while the feature is broken for the end user (“PASS-bluffs” and “FAIL-bluffs”). Each forensic anchor in the Constitution records a real incident (e.g. the 2026-05-20 D3 audio-routing PASS-bluff where validation went green with an empty “Codec In Use” field, or the 2026-06-25 giant-button UI that passed token-equality tests while the real screen was broken). The Constitution exists to make that whole class of dishonest success mechanically impossible, once, universally — so the discipline cannot drift between projects or be quietly forgotten.

Why it's a game-changer: It converts engineering culture from documentation-people-hope-to-follow into inherited, versioned, mechanically-enforced law. One submodule bump upgrades the rules for the entire fleet; a single anti-bluff covenant is guaranteed present in every consuming repo because a propagation gate literally greps for the clause number across the fleet and a paired mutation test proves the gate itself is not bluffing. Governance becomes an auditable fact, not an aspiration.

What's innovative: - Constitution-as-submodule — engineering law distributed and version-pinned exactly like code, with deliberate v1.0.0-style tags and per-project pinning. - **Anti-bluff as a first-class, forensic doctrine** — every clause traces to a verbatim operator mandate and, often, the exact incident that motivated it. - **Meta-testing of the rules themselves (§1.1)** — every gate is paired with a mutation that must flip PASS→FAIL, so “the gate isn't a sham” is itself proven. - **Earned universality** — an explicit four-part test decides whether a rule is universal or project-specific, keeping the base clean.

How it is used across all products (the powers it gives): - Governance backbone: every Helix/vasic-digital project adds it as a submodule and imports it from CLAUDE.md / AGENTS.md / QWEN.md / its own Constitution.md; the rules apply unconditionally. - **Gates & mandates:** it defines the four-layer coverage model (source-present, survives-build, behaves-at-runtime, gate-not-bluffing) plus named mandates — credentials handling (§11.4.10), documentation always-sync (§11.4.60), containers-submodule mandate (§11.4.76), CodeGraph (§11.4.78), mandatory test-type coverage (§11.4.169), and more. - **Propagation:** CM-COVENANT-114-NNN-PROPAGATION gates assert the literal clause text is present across the consuming fleet; non-compliance is a release blocker with no escape-hatch flags. - **Discovery:** submodules-catalogue.md

makes “do we already own something that does X?” a one-glance answer before any new module is scaffolded. - **Consistency of AI agents:** the same law is expressed to every CLI agent (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi via AGENTS.md, Qwen Code via QWEN.md), so all tooling obeys one covenant.

Biggest challenges + how solved: - *Locating the submodule from arbitrary nested depth* → `find_constitution.sh` walks up parent directories and follows the git superproject pointer recursively, with a `CONSTITUTION_DIR` override and two supported layouts (`constitution/`, `submodules/constitution/`). - *Keeping one repo authoritative across four Git providers* → `install_upstreams.sh` reads `Upstreams/*.sh` and configures origin with multiple push URLs so a single git push fans out to GitHub (primary), GitLab, GitFlic, and GitVerse. - *Preventing rule-bloat / project leakage into the universal base* → the earned-universality four-part test plus §11.4.17 universal-vs-project classification on every new rule. - *Proving the inheritance gate actually works* → `meta_test_inheritance.sh`, a sentinel meta-test that deletes the §11.4 anchor and asserts the gate catches it.

Tech/mechanism stack (why + how): - **Git-submodule inheritance** — *why:* one authoritative rulebook, version-pinned per project; *how:* consuming projects add the submodule and @import its agent files; three-layer top-to-bottom evaluation, project extends-not-weakens. - **find_constitution.sh** — *why:* nested submodules must source the rules without knowing their own depth; *how:* parent-walk + `git rev-parse --show-superproject-working-tree` recursion + `CONSTITUTION_DIR` override. - **install_upstreams.sh + Upstreams/** — *why:* four-provider redundancy with one push; *how:* declarative `.sh` remotes materialised into multi-URL origin. - **§1.1 mutation meta-tests** — *why:* a gate that can never fail is worse than none; *how:* each gate paired with a sed-out/rename mutation that must turn PASS→FAIL, then restore. - **Propagation gates (CM-COVENANT-114-NNN-PROPAGATION)** — *why:* guarantee a clause is actually present fleet-wide; *how:* literal clause-number grep across consumers + paired §1.1 mutation. - **submodules-catalogue.md (§11.4.74)** — *why:* stop re-implementing owned capabilities; *how:* 142-repo capability-grouped inventory, catalogue-check recorded in the tracker before scaffolding. - **Multi-format export** — *why:* human + tooling + archival consumption; *how:* every canonical doc emitted as `.md` / `.html` / `.pdf` / `.docx`.

Public links: - Canonical (public): <https://github.com/HelixDevelopment/HelixConstitution> - Mirror (public): <https://github.com/vasic-digital/HelixConstitution> - Additional upstream mirrors: GitLab `helixdevelopment1/helixconstitution`, GitFlic `helixdevelopment/helixconstitution`, GitVerse `helixdevelopment/HelixConstitution`.

Suggested diagrams/illustrations (OpenDesign): 1.

Constitution inheritance layers — a three-tier stack (universal base submodule → project layer → subdirectory overrides) with arrows showing “extend, never weaken.” 2. **Fleet propagation** — one Constitution repo at the centre, submodule links radiating to 140+ consuming repos, each stamped with a green CM-COVENANT-...-PROPAGATION check. 3. **Anti-bluff evidence pipeline** — code change → four-layer gate (source / build / runtime / mutation) → captured-evidence artefact → PASS, with a “no evidence = bluff = blocker” reject branch. 4. **Multi-upstream push topology** — a single git push fanning out to GitHub / GitLab / GitFlic / GitVerse.

Site relevance: both

Priority tier: Helix-primary (governance pillar)

Source provenance: - /Volumes/T7/Projects/constitution/README.md (inheritance model, consumption steps, multi-upstream topology, contents table, earned-universality test). - /Volumes/T7/Projects/constitution/Constitution.md — structural read: §1.1 (lines 147-164, false-positive immunity / mutation pairing), §11.4.6 (630-649, no-guessing), §11.4.69 (6411+, positive-evidence taxonomy + D3 forensic incident), §11.4.169 (9787+), §11.4.141 (9851+), §11.4.170 (giant-button forensic anchor), ToC (lines 20-110). - /Volumes/T7/Projects/constitution/find_constitution.sh (parent-walk + superproject recursion + CONSTITUTION_DIR override). - /Volumes/T7/Projects/constitution/submodules-catalogue.md (142 repos, catalogue-first §11.4.74). - /Volumes/T7/Projects/vasic/_analysis/github-helix-others.md and github-vasic-digital.md (repo visibility = public, descriptions, ecosystem role).